

University
Politehnica
of Bucharest





PostgreSQL

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@cs.pub.ro



Overview

- PostgreSQL Overview
- PostgreSQL SQL



Overview

- PostgreSQL Overview
- PostgreSQL SQL



PostgreSQL Overview

- PostgreSQL 12
- PostgreSQL is an object-relational database management system (ORDBMS) based on **POSTGRES, Version 4.2**, developed at the University of California at Berkeley Computer Science Department
- POSTGRES pioneered many concepts that only became available in some commercial database systems much later
- PostgreSQL is an open-source descendant of the original Berkeley code
- PostgreSQL can be used, modified, and distributed by anyone free of charge for any purpose, be it private, commercial, or academic



PostgreSQL Overview

- PostgreSQL supports a large part of the SQL standard and offers many modern features:
  - Complex queries
  - Foreign keys
  - Triggers
  - Updatable views
  - Transactional integrity
  - Multiversion concurrency control

- PostgreSQL can be extended by the user in many ways, for example by adding new
  - Data types
  - Functions
  - Operators
  - Aggregate functions
  - Index methods
  - Procedural languages



PostgreSQL Architecture (Simple)

- PostgreSQL uses a client/server model

- A PostgreSQL session consists of the following cooperating processes (programs):
  - A server process
  - A client application

- The server process
  - Manages the database files
  - Accepts connections to the database from client applications (**postgremaster**)
  - Performs database actions on behalf of the clients
  - The database server program is called **postgres**.



PostgreSQL Architecture (Simple)

- Client applications can be very diverse in nature
- A client could be
  - A text-oriented tool
  - A graphical application
  - A web server that accesses the database to display web pages
  - A specialized database maintenance tool
- Some client applications are supplied with the PostgreSQL distribution
- Most client applications are developed by users

PostgreSQL Architecture (Simple)

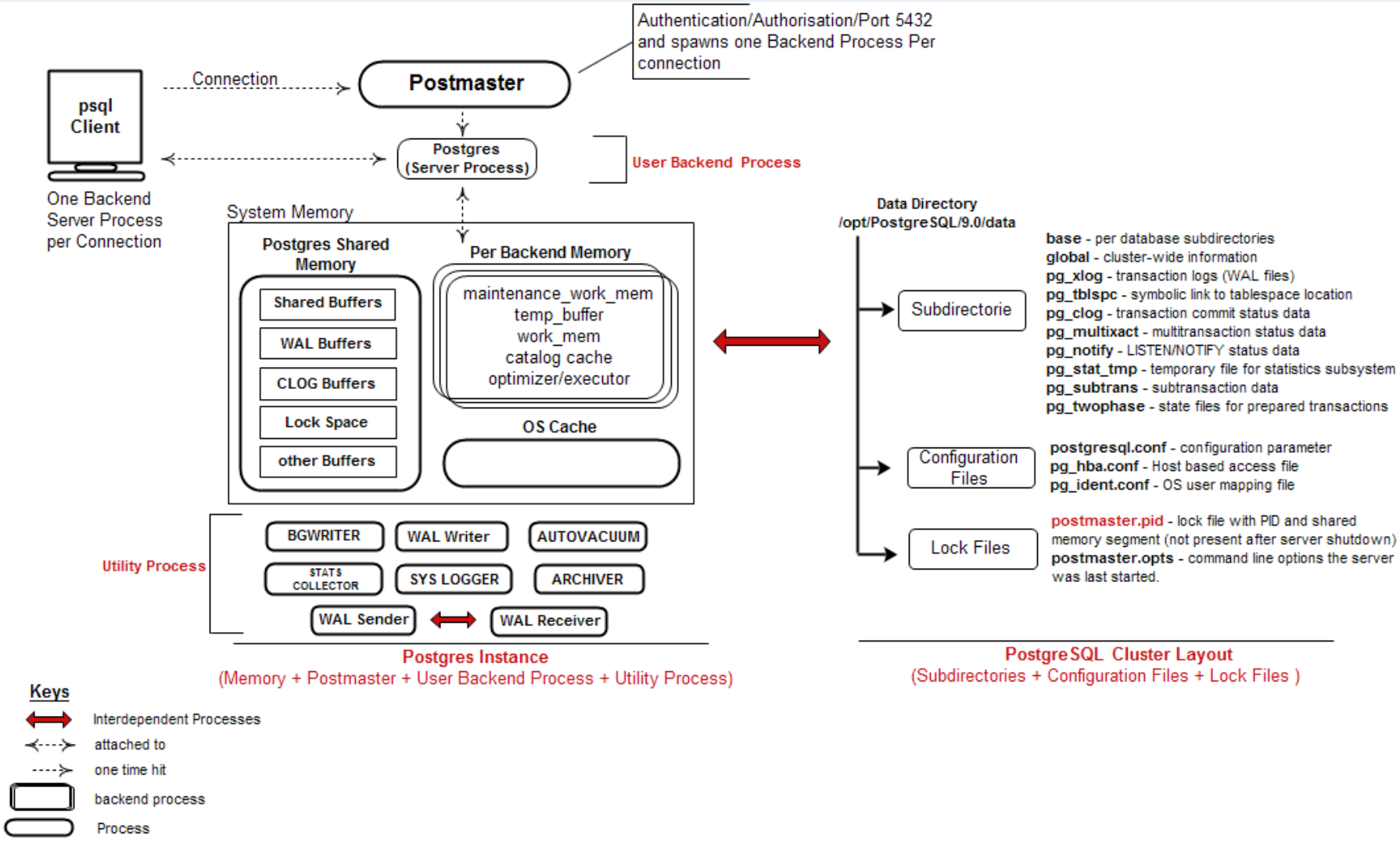
- The client and the server can be on different hosts (as is typical of client/server applications)
- The communicate between the client and server happens over a TCP/IP network connection
- Note: The files that can be accessed on a client machine might not be accessible on the database server machine



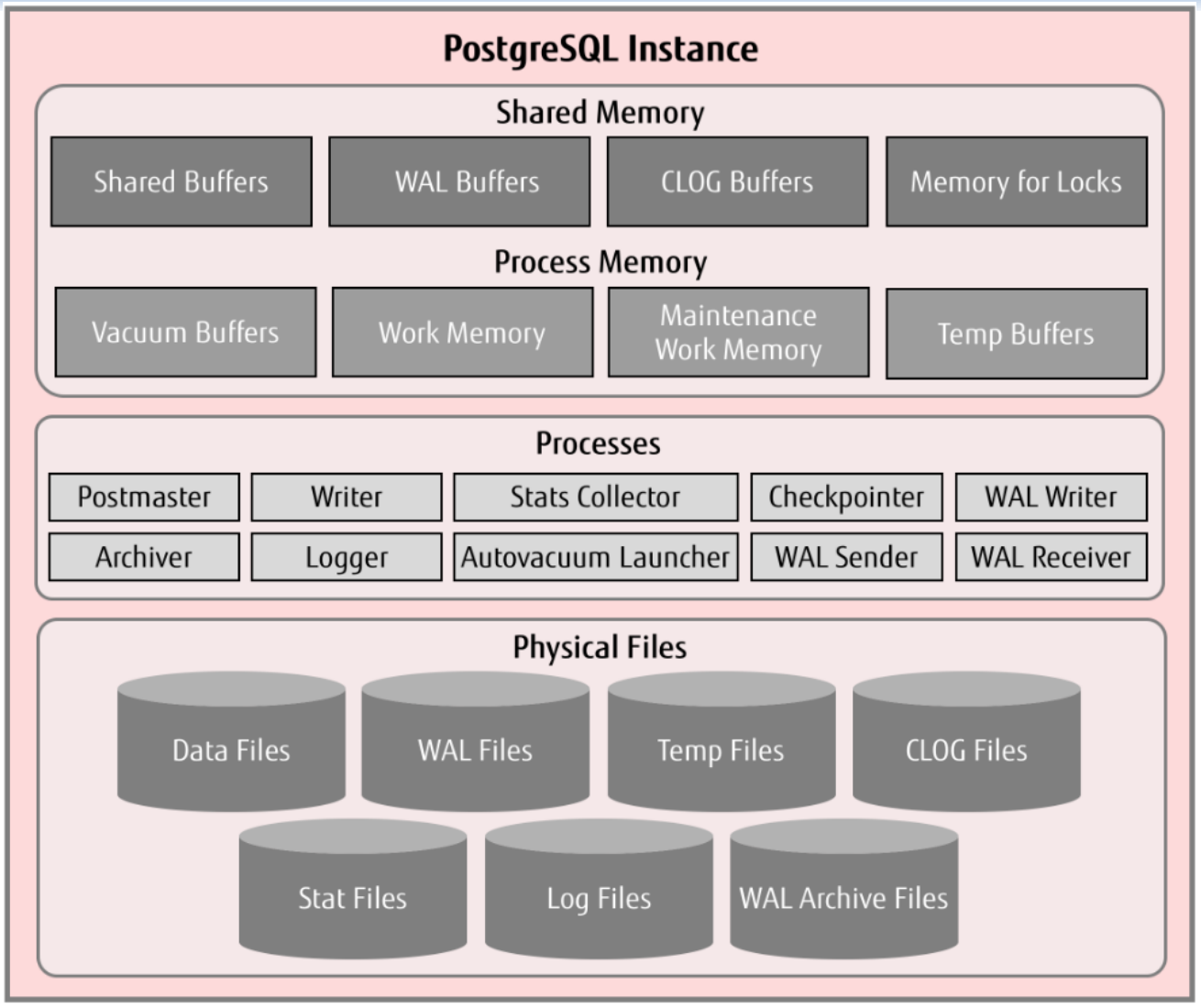
PostgreSQL Architecture (Simple)

- The PostgreSQL server can handle multiple concurrent connections from clients
  - To achieve this it starts ("forks") a new process for each connection
  - After the "fork" is started, the client and the new server process communicate without intervention by the original **postgres** process
  - The master server process (**postmaster**) is always running, waiting for client connections
  - The clients and associated server processes come and go

PostgreSQL Architecture



PostgreSQL Architecture





PostgreSQL Architecture

- A PostgreSQL instance consists of a set of Processes and Memory Buffers
- PostgreSQL uses a simple "process per-user" client/server model
- PostgreSQL types of process:
  - The **'postmaster'**
  - Utility processes (bgwriter, walwriter, syslogger, archiver, statscollector, autovacuum lancher)
  - User Backend process (**postgres** process itself, Server Process)
- The postmaster:
  - Is a supervisory daemon process
  - Is attached to shmmem segment but refrains from accessing it



PostgreSQL Architecture

- When a client request for connection to the database
  - The first request is sent to the **postmaster** daemon process
  - After performing Authentication and authorization it spawns one new backend server process (**postgres**)
  - From this point on, the frontend process and the backend server communicate without intervention by the **postmaster**
    - The postmaster is always running, waiting for connection requests
    - The frontend and backend processes come and go
  - The **libpq** library allows a single frontend to make multiple connections to backend processes

- **postmaster** and **postgres** servers run with the user ID of the PostgreSQL "superuser"



PostgreSQL Architecture

- One **postgres** process exists for every open database session
- Once a user is authenticated with a user connection, it directly connects with the shared memory
- The communication over any one frontend-to-backend connection is single-threaded
- Each backend process is a single-threaded process that can only execute one query at a time



PostgreSQL Architecture

- PostgreSQL shared memory
  - Is very big
  - The buffers are not in sync (i.e., the buffers work independent)

- Buffers
  - Shared Buffers
  - WAL Buffers
  - CLOG Buffers
  - Other Buffers



PostgreSQL Architecture
Shared Buffers

- Shared buffers defines a block of memory that PostgreSQL will use to hold requests that are awaiting attention from the kernel buffer and CPU
- The default value is quite low for any real-world workload and needs to be adjusted through the **shared_buffers** parameter
- Unlike databases like Oracle, more is not always better
  - There is a threshold above which increasing this value can hurt performance
- The shared buffers represent the area of memory PostgreSQL uses to perform work
  - It should be large enough to handle work-load on database server
  - Otherwise PostgreSQL will start pushing data to file and it will hurt the performance overall



PostgreSQL Architecture
Shared Buffers

- The value of the shared buffers should be set based on
  - The dataset size which the database server is supposed to handle at peak loads and
  - The available RAM (Note: RAM used by other applications on the server is not available)
  - The default is typically 128MB, but might be less if your kernel settings will not support it (as determined during **initdb**)
  - Settings significantly higher values than the minimum are usually needed for good performance
  - The *shared_buffers* parameter can only be set at server start
- A reasonable starting value for *shared_buffers* is 25% of the memory in your system
- Set the value of *shared_buffers* in the *postgresql.conf* file



PostgreSQL Architecture
Shared Buffers

- There are some workloads where even larger settings for shared_buffers are effective
  - It is unlikely that an allocation of more than 40% of RAM to shared_buffers will work better than a smaller amount because PostgreSQL also relies on the operating system cache
- Larger settings for shared_buffers usually require a corresponding increase in max_wal_size
  - To spread out the process of writing large quantities of new or changed data over a longer period of time



PostgreSQL Architecture
WAL Buffers

- WAL (**Write-Ahead Log**) is an optimization PostgreSQL uses to minimize disk I/O while still preventing data loss
- The write ahead log (WAL) buffers are also called "transaction log buffers"
- The WAL buffers represent the amount of memory allocation for storing WAL data
- The WAL data
  - Is the metadata information about changes to the actual data
  - Is enough to reconstruct actual data during database recovery operations
  - Is written to a set of physical files in persistent location called "WAL segments" or "checkpoint segments"



PostgreSQL Architecture
WAL

- Whenever a transaction completes
  - A record of every single change that transaction made must have been written out to persistent storage
  - The changes that need to be written out include every single row that was inserted, deleted, or updated
  - If transactions completed before writing all the changes made to disk and the power went out before they were, Postgres would suffer data loss
  - When Postgres would come back up, it will see the transaction completed, but Postgres will have no record of some of the changes made by the transaction



PostgreSQL Architecture
WAL

- The naive way to make sure all changes are written to disk is to perform those changes on disk:
  - Every time a transaction modifies a row it would perform a write to the location of that row on disk
  - The downside to this approach is it requires performing a lot of random writes to disk, one to each row that was changed by the transaction
  - That many random writes winds up being inefficient
  - To reduce the amount of necessary I/O, Postgres instead uses write-ahead logging



PostgreSQL Architecture
WAL

- When using a WAL, a transaction must do two things to modify a row:
  1. It modifies a copy of the row in memory
     - Queries running know to first check if there is a copy of the row in memory first
     - Queries will always use the new copy instead of using the old copy on disk
  2. A record describing the change made is added to the "WAL log"
     - The description logs what the change was and where
     - The WAL log is simply a record of all changes that have been made
     - Is stored on disk



PostgreSQL Architecture
WAL

- WAL prevents data loss
  - The WAL log can be used to restore the proper state of the database
  - A transaction must make sure to flush the WAL log to disk before completing
  - Thus, there is a record of all changes made by that transaction on disk

- When Postgres is coming back up from a crash
  1. Goes through all the records in the WAL log that may not have been propagated to disk
  2. Performs those changes if they were not performed

- The WAL also performs less disk I/O because writes to the WAL log can be
  - Batched together
  - Flushed to disk in a single write
  - The total amount of disk I/O needed is dramatically reduced if the WAL writes multiple records to disk at the same time



PostgreSQL Architecture
WAL Parameters

- The following parameters are in the *postgresql.conf*

- *wal_level*
  - Determines how much information is written to the WAL
  - This parameter can only be set at server start
  - Value:
    - *replica* (default) writes enough data to support
      - WAL archiving and replication
      - Running read-only queries on a standby server
    - *minimal* removes all logging except the information required to recover from a crash or immediate shutdown
    - *logical* adds information necessary to support logical decoding



PostgreSQL Architecture
WAL Parameters

- *fsync* deals with how the data is physically written on disk

- *fsync* On
  - The PostgreSQL server will try to make sure that updates are physically written to disk
  - It issues the *fsync()* system calls or various equivalent methods
  - Ensures that the database cluster can recover to a consistent state after an operating system or hardware crash

- *fsync* Off
  - Increases performance
  - Can result in unrecoverable data corruption in the event of a power failure or system crash
  - Use it when the entire database can be easily recreated from external data



PostgreSQL Architecture
WAL Parameters

- ***wal_sync_method***
  - Method used for forcing WAL updates out to disk
  - If *fsync* is off then this setting is irrelevant
    - WAL file updates will not be forced out at all
  - Possible values are:
    - *open_datasync* (write WAL files with *open()* option O_DSYNC)
    - *fdatasync* (call *fdatasync()* at each commit)
    - *fsync* (call *fsync()* at each commit)
    - *fsync_writethrough* (call *fsync()* at each commit, forcing write-through of any disk write cache)
    - *open_sync* (write WAL files with *open()* option O_SYNC)

- ***O_DSYNC***
  - Write operations on the file will complete according to the requirements of synchronized I/O data integrity completion
  - This provides the guarantee that when the system call returns the file data is on disk
- ***O_SYNC***
  - Write operations on the file will complete according to the requirements of synchronized I/O file integrity completion
  - Guarantees not only that the file data has been written to disk but also the file metadata



PostgreSQL Architecture
WAL Parameters

- ***wal_buffers***
  - The amount of shared memory used for WAL data that has not yet been written to disk
  - The default setting of -1 selects a size equal to 1/32nd (about 3%) of shared_buffers:
    - Not less than 64KB
    - Not more than the size of one WAL segment (typically 16MB)
  - The contents of the WAL buffers are written out to disk at every transaction commit
  - Extremely large values are unlikely to provide a significant benefit
  - Setting this value to at least a few megabytes can improve write performance on a busy server where many clients are committing at once
  - The auto-tuning selected by the default setting of -1 should give reasonable results in most cases



PostgreSQL Architecture Checkpointer

- The background job called the checkpointer is the job that writes data to disk
- The checkpointer is constantly
  - Going through all the changes made in memory and not on disk yet
  - Performing those changes on disk (writing changes to disk)
- The checkpointer can perform changes to multiple rows near each other on disk at the same time



PostgreSQL CLOG Buffers

- CLOG stands for "commit log"
- The CLOG buffers is an area in operating system RAM dedicated to hold commit log pages
- The commit log pages contain log of transaction metadata (differ from the WAL data)
- The commit logs
  - Have the commit status of all transactions
  - Indicate whether or not a transaction has been completed (committed)
- There is no specific parameter to control this area of memory
- This is automatically managed by the database engine in tiny amounts
- This is a shared memory component: is accessible to all the background server and user processes of a PostgreSQL database

PostgreSQL
Other Buffers

- Memory for Locks / Lock Space:
  - Store all heavyweight locks used by the PostgreSQL instance
  - Is shared across all the background server and user processes connecting to the database

- Vacuum Buffers
  - Is the maximum amount of memory used by each of the *AUTOVACUUM* worker processes
  - *AUTOVACUUM* daemon is an optional feature that automatically vacuums the database so that you don't have to manually run the *VACUUM* statement
  - The *VACUUM* statement is used to reclaim storage by removing obsolete data or tuples from the PostgreSQL database



PostgreSQL Other Buffers

- Work Memory
  - This is the amount of memory reserved for either a single **sort** or **hash table** operation in a query
  - Is controlled by **work_mem** database parameter
  - A sort operation could be one of: **ORDER BY**, **DISTINCT** or **Merge join**
  - A **hash table** operation could be due to a **hash-join**, **hash-based aggregation** or an **IN** subquery

- Maintenance Work Memory
  - This is the maximum amount of memory allocation of RAM consumed for maintenance operations
  - A maintenance operation could be:
    - **VACUUM**
    - **CREATE INDEX**
    - adding a **FOREIGN KEY** to a table



PostgreSQL
Other Buffers

- Temp Buffers
  - A database may have one or more temporary tables
  - The data blocks (pages) of such temporary tables need a separate allocation of memory to be processed in
  - The temp buffers serve this purpose by utilizing a portion of RAM, defined by the **temp_buffers** parameter
  - The temp buffers are only used for access to temporary tables in a user session



Overview

- PostgreSQL Overview
- PostgreSQL SQL Language



PostgreSQL
SQL Language

- PostgreSQL as a relational database management system (RDBMS)
- It stores data into tables
  - Each table is a named collection of rows
  - Each row of a given table has the same set of named columns
  - Each column is of a specific data type
- Columns have a fixed order in each row
- Rows do not have a fixed order in a table
- SQL does not guarantee the order of the rows within the table in any way
- Tables are grouped into databases



PostgreSQL
SQL Language

- PostgreSQL support the standard SQL Language
  - Data Manipulation Language (DML)
  - Data Definition Language (DDL)
  - Data Control Language (DCL)
  - Transaction Control Language (TCL)

- Full support of numeric, string, date/time, XML, JSON, etc. data types

- Full support of different indexes

- Supports Full Text Search
  - Indexes
  - Search operations
  - Parsers
  - Dictionaries, e.g., Stop Words



PostgreSQL
DDL

- To create a new table or object use the CREATE command
  - CREATE TABLE names AS SELECT first_name, last_name FROM employees;
  - CREATE TABLE IF NOT EXISTS names AS SELECT first_name, last_name FROM employees;
  - CREATE OR REPLACE VIEW v_names AS SELECT first_name, last_name FROM employees;
- To delete an existing object use the DROP command
  - DROP TABLE names;
  - DROP TABLE IF EXISTS names;
  - DROP VIEW v_names;
  - DROP VIEW IF EXISTS v_names;



PostgreSQL DDL

- To modify a table use the ALTER command
- Add a column:
  - ALTER TABLE names ADD COLUMN salary numeric(10,2);
- Add a column with constraint:
  - ALTER TABLE names ADD COLUMN salary numeric(10,2) CHECK (salary > 0);
- Remove a column
  - ALTER TABLE names DROP COLUMN salary;
- Remove a column with CASCADE to drop everything that depends on that column
  - ALTER TABLE names DROP COLUMN salary CASCADE;



PostgreSQL
DDL

- Add constraints
  - ALTER TABLE names ADD CHECK (salary > 0);
  - ALTER TABLE names ADD CONSTRAINT some_name UNIQUE (last_name, first_name);
  - ALTER TABLE names ALTER COLUMN last_name SET NOT NULL;
- Remove constraints
  - ALTER TABLE names DROP CONSTRAINT some_name;
  - ALTER TABLE names ALTER COLUMN last_name DROP NOT NULL;
- Change default values
  - ALTER TABLE names ALTER COLUMN salary SET DEFAULT 1000.0;
  - ALTER TABLE names ALTER COLUMN salary DROP DEFAULT;



PostgreSQL
DDL

- Change column data types
  - ALTER TABLE names ALTER COLUMN salary TYPE numeric(6,2);
- Rename columns
  - ALTER TABLE names RENAME COLUMN salary TO sal;
- Rename tables
  - ALTER TABLE names RENAME TO emp_names;
- Delete all rows from a table
  - TRUNCATE TABLE names;



PostgreSQL Privileges

- When an object is created, it is assigned an owner
- The owner is normally the role that executed the creation statement
- For most kinds of objects, the initial state is that only the owner (or a superuser) can do anything with the object
- To allow other roles to use it, privileges must be granted
- There are different kinds of privileges: SELECT, INSERT, UPDATE, DELETE, TRUNCATE, REFERENCES, TRIGGER, CREATE, CONNECT, TEMPORARY, EXECUTE, and USAGE
- The privileges applicable to an object vary depending on the object's type (table, function, etc)



PostgreSQL Privileges

- An object can be assigned to a new owner with an ALTER command of the appropriate kind for the object
- E.g. :

  ALTER TABLE *table_name* OWNER TO *new_owner*;

- Superusers can always do this
- Ordinary roles can only do it if they are both
  - The current owner of the object or a member of the owning role
  - A member of the new owning role



PostgreSQL
DCL

- To assign privileges, the GRANT command is used:

GRANT {privilage_name | ALL} ON object_name TO {user_name | PUBLIC};

- Writing ALL in place of a specific privilege grants all privileges that are relevant for the object type
- The special "role" name PUBLIC can be used to grant a privilege to every role on the system.
- To revoke a privilege, use the fittingly named REVOKE command:

REVOKE {privilage_name | ALL} ON object_name FROM {user_name | PUBLIC};



PostgreSQL Inheritance

- PostgreSQL implements table inheritance
- Table inheritance is typically established when the child table is created, using the INHERITS clause of the CREATE TABLE statement

```
1
2  CREATE TABLE cities2 AS SELECT * FROM cities
3  WHERE city_id NOT IN (4, 5, 8, 9, 11, 12, 15, 17, 23);
4
5
6  CREATE TABLE capitals(
7      capital boolean DEFAULT true
8  ) INHERITS (cities2);
9
10 INSERT INTO capitals SELECT * FROM cities
11 WHERE city_id IN (4, 5, 8, 9, 11, 12, 15, 17, 23);
12
13 SELECT city FROM cities2;
14
15 SELECT city FROM ONLY cities2;
```



PostgreSQL Inheritance

- A child-parent relationship can be added to existing tables using ALTER TABLE with the INHERIT

ALTER TABLE capitals NO INHERIT cities2;
ALTER TABLE capitals INHERIT cities2;

- To do this the new child table must already include columns with the same names and types as the columns of the parent
- It must also include check constraints with the same names and check expressions as those of the parent
- Similarly, an inheritance link can be removed from a child using the NO INHERIT variant of ALTER TABLE
- Dynamically adding and removing inheritance links like this can be useful when the inheritance relationship is being used for table partitioning



PostgreSQL Partitioning

- Partitioning refers to splitting what is logically one large table into smaller physical pieces.

- Partitioning benefits

1. Query performance can be improved dramatically in certain situations:
    - Particularly when most of the heavily accessed rows of the table are in a single partition or a small number of partitions
    - The partitioning substitutes for leading columns of indexes, reducing index size and making it more likely that the heavily-used parts of the indexes fit in memory

2. When queries or updates access a large percentage of a single partition performance can be improved by taking advantage of a sequential scan of that partition instead of using an index and random-access reads scattered across the whole table



PostgreSQL Partitioning

- Partitioning benefits

3. Bulk loads and deletes can be accomplished by adding or removing partitions if that requirement is planned into the partitioning design
    - Doing ALTER TABLE DETACH PARTITION or dropping an individual partition using DROP TABLE is far faster than a bulk operation
    - These commands also entirely avoid the VACUUM overhead caused by a bulk DELETE

4. Seldom-used data can be migrated to cheaper and slower storage media.



PostgreSQL Partitioning

- The benefits will normally be worthwhile only when a table would otherwise be very large
- The exact point at which a table will benefit from partitioning depends on the application
- A rule of thumb is that the size of the table should exceed the physical memory of the database server.



PostgreSQL Partitioning Types

- Range Partitioning
  - The table is partitioned into “ranges” defined by a key column or set of columns
  - There is no overlap between the ranges of values assigned to different partitions
- List Partitioning
  - The table is partitioned by explicitly listing which key values appear in each partition.
- Hash Partitioning
  - The table is partitioned by specifying a modulus and a remainder for each partition
  - Each partition will hold the rows for which the hash value of the partition key divided by the specified modulus will produce the specified remainder



PostgreSQL Declarative Partitioning

- PostgreSQL offers a way to specify how to divide a table into pieces called partitions
- The table that is divided is referred to as a *partitioned table*
- The specification consists of
  - The *partitioning method*
  - A list of columns or expressions to be used as the *partition key*
- All rows inserted into a partitioned table will be routed to one of the *partitions* based on the value of the partition key
- Each partition has a subset of the data defined by its *partition bounds*
- Partitions may themselves be defined as partitioned tables (using what is called *sub-partitioning)*



PostgreSQL Declarative Partitioning

- Partitions may have
  - Indexes
  - Constraints
  - Default values (distinct from those of other partitions)

- It is not possible to turn a regular table into a partitioned table or vice versa

- It is possible to
  - Add a regular or partitioned table containing data as a partition of a partitioned table
  - Remove a partition from a partitioned table turning it into a standalone table
  - Use ALTER TABLE to with ATTACH PARTITION and DETACH PARTITION sub-commands.



PostgreSQL
Declarative Partitioning

```sql
CREATE TABLE sensors(
    sensor_id       int not null,
    reading_time    timestamp not null,
    humidity        numeric(6,2),
    temperature     numeric(6,2)
) PARTITION BY RANGE (reading_time);

CREATE TABLE sensors_y2019q1 PARTITION OF sensors
FOR VALUES FROM ('2019-01-01') TO ('2019-04-01');

CREATE TABLE sensors_y2019q2 PARTITION OF sensors
FOR VALUES FROM ('2019-04-01') TO ('2019-07-01');

CREATE TABLE sensors_y2019q3 PARTITION OF sensors
FOR VALUES FROM ('2019-07-01') TO ('2019-10-01');

CREATE TABLE sensors_y2019q4 PARTITION OF sensors
FOR VALUES FROM ('2019-10-01') TO ('2020-01-01');

-- subpartitioning
CREATE TABLE sensors_y2019m01hum PARTITION OF sensors
FOR VALUES FROM ('2006-01-01') TO ('2006-02-01')
PARTITION BY RANGE (humidity);
```



PostgreSQL Declarative Partitioning

- The following limitations apply to partitioned tables:
  - There is no way to create an exclusion constraint spanning all partitions
    - It is only possible to constrain each leaf partition individually
  - Unique constraints on partitioned tables must include all the partition key columns.
    - This limitation exists because PostgreSQL can only enforce uniqueness in each partition individually.
  - BEFORE ROW triggers, if necessary, must be defined on individual partitions, not the partitioned table
  - Mixing temporary and permanent relations in the same partition tree is not allowed.
    - If the partitioned table is permanent, so must be its partitions and likewise if the partitioned table is temporary
    - When using temporary relations, all members of the partition tree must be from the same session



PostgreSQL
Partitioning Using Inheritance

- While the built-in declarative partitioning is suitable for most common use cases, there are some circumstances where a more flexible approach may be useful

- Partitioning can be implemented using table inheritance, which allows for several features not supported by declarative partitioning

Declarative partitioning	Inheritance partitioning
Partitions must have the same set of columns as the partitioned table for declarative partitioning	Child tables may have extra columns not present in the parent with table inheritance
Doesnot Allows for multiple inheritance	Allows for multiple inheritance
Declarative partitioning only supports range, list and hash partitioning	Allows data to be divided in a manner of the user's choosing
Some operations require a stronger locks. E.g. Adding or removing a partition to or from a partitioned table requires taking an ACCESS EXCLUSIVE lock on the parent table	Some operations don't require a stronger locks. E.g. Adding or removing a partition to or from a partitioned table requires taking a SHARE UPDATE EXCLUSIVE lock in the case of regular inheritance



PostgreSQL
Partitioning Using Inheritance

```sql
CREATE TABLE sensors(
    sensor_id       int not null,
    reading_time    timestamp not null,
    humidity        numeric(6,2),
    temperature     numeric(6,2)
);

CREATE TABLE sensors_y2019q1(
    CHECK ( reading_time >= DATE '2019-01-01' AND reading_time < DATE '2019-04-01' )
) INHERITS (sensors);

CREATE TABLE sensors_y2019q2(
    CHECK ( reading_time >= DATE '2019-04-01' AND reading_time < DATE '2019-07-01' )
) INHERITS (sensors);

CREATE TABLE sensors_y2019q3(
    CHECK ( reading_time >= DATE '2019-07-01' AND reading_time < DATE '2019-10-01' )
) INHERITS (sensors);

CREATE TABLE sensors_y2019q4(
    CHECK ( reading_time >= DATE '2019-10-01' AND reading_time < DATE '2020-01-01' )
) INHERITS (sensors);
```



PostgreSQL
Partitioning Using Inheritance

```
CREATE OR REPLACE FUNCTION function_insert_sensors()
RETURNS TRIGGER AS $$
BEGIN
    IF    ( NEW.reading_time >= DATE '2019-01-01' AND
            NEW.reading_time < DATE '2019-04-01' ) THEN
        INSERT INTO sensors_y2019q1 VALUES (NEW.*);
    ELSIF ( NEW.reading_time >= DATE '2019-04-01' AND
            NEW.reading_time < DATE '2019-07-01' ) THEN
        INSERT INTO sensors_y2019q2 VALUES (NEW.*);
    ELSIF ( NEW.reading_time >= DATE '2019-07-01' AND
            NEW.reading_time < DATE '2019-10-01' ) THEN
        INSERT INTO sensors_y2019q3 VALUES (NEW.*);
    ELSIF ( NEW.reading_time >= DATE '2019-10-01' AND
            NEW.reading_time < DATE '2020-01-01' ) THEN
        INSERT INTO sensors_y2019q4 VALUES (NEW.*);
    ELSE
        RAISE EXCEPTION 'Date out of range!';
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_insert_sensors
BEFORE INSERT ON sensors
FOR EACH ROW EXECUTE FUNCTION function_insert_sensors();

INSERT INTO sensors VALUES(1, now(), 34, 29);
```



PostgreSQL
Data manipulation – Querying Data

- The process of retrieving or the command to retrieve data from a database is called a query
- In SQL, the SELECT command is used to specify queries
- The general syntax of the SELECT command is

[WITH with_queries] SELECT select_list FROM table_expression [sort_specification]

- PosrgreSQL implements the standardized SELECT clause

```
1 WITH q_info as (
2     SELECT country_id, country_name, region_name
3     FROM countries NATURAL JOIN REGIONS
4 )
5 SELECT city, country_name, region_name
6 FROM cities NATURAL JOIN q_info;
```



PostgreSQL
Data manipulation – Querying Data

```sql
SELECT c.city, i.country_name, i.region_name
FROM cities c, (SELECT country_id, country_name, region_name
                FROM countries NATURAL JOIN regions
                WHERE country_id=c.country_id) i;

SELECT c.city, i.country_name, i.region_name
FROM cities c, (SELECT country_id, country_name, region_name
                FROM countries NATURAL JOIN regions) i
WHERE i.country_id=c.country_id;
```



PostgreSQL
Data manipulation – Querying Data

- Subqueries appearing in FROM can be preceded by the key word LATERAL
- This allows them to reference columns provided by preceding FROM items

```
1 SELECT c.city, i.country_name, i.region_name
2 FROM cities c, LATERAL(SELECT country_id, country_name, region_name
3                        FROM countries NATURAL JOIN regions
4                        WHERE country_id=c.country_id) i;
```



PostgreSQL
Data manipulation – Inserting Data

- To create a new row, use the INSERT command
- The INSERT command respects the standard syntax

```
1 INSERT INTO regions VALUES(5, 'Oceania');
2
3 INSERT INTO cities(city_id, city, country_id) VALUES(30, 'Copenhaga', 'DK');
4
5 CREATE TABLE city_names(city  varchar(30));
6
7 INSERT INTO city_names SELECT city FROM cities;
```

- INSERT can be used to insert multiple data with and without specifing the column name

```
1 INSERT INTO countries VALUES ('RO', 'Romania', 1), ('ME', 'Moldova', 1);
```



PostgreSQL
Data manipulation – Updating Data

- To update existing rows, use the UPDATE command. This requires three pieces of information:

1. The name of the table and column to update
2. The new value of the column
3. Which row(s) to update

```
1 UPDATE countries
2 SET country_id='MD'
3 WHERE country_id='ME';
```



PostgreSQL
Data manipulation – Deleting Data

- You use the DELETE command to remove rows

```
1 DELETE FROM countries
2 WHERE country_id IN ('RO', 'MD');
```



PostgreSQL
Returning data from modified rows

- Sometimes it is useful to obtain data from modified rows while they are being manipulated
- The INSERT, UPDATE, and DELETE commands all have an optional RETURNING clause that supports this
- Use of RETURNING
- Avoids performing an extra database query to collect the data
- Is especially valuable when it would otherwise be difficult to identify the modified rows reliably
- The RETURNING clause can return individual columns or all columns using *



PostgreSQL
Returning data from modified rows

```sql
UPDATE countries SET country_id='MD'
WHERE country_id='ME' RETURNING region_id;

INSERT INTO cities(city_id, city, country_id)
VALUES (31, 'Bucuresti', 'RO'), (32, 'Chisinau', 'MD')
RETURNING *;

WITH q AS (
    DELETE FROM cities WHERE country_id IN ('RO', 'MD')
    RETURNING city
)
INSERT INTO city_names SELECT * FROM q RETURNING city;
```



PostgreSQL Data Types

- Numeric data types:
  - Integer Types: **SMALLINT**, **INTEGER**, **BIGINT**
  - Arbitrary Precision Numbers: **DECIMAL**, **NUMERIC(*precision*, *scale*)**
  - Floating-Point Types: **REAL**, **DOUBLE PRECISION**
  - Serial types: **SMALLSERIAL**, **SERIAL**, **BIGSERIAL**
    - A notational convenience for creating unique identifier columns
    - Are implemented using sequences
- Monetary Types：**MONEY**
  - The money type stores a currency amount with a fixed fractional precision



PostgreSQL Data Types

```sql
CREATE TABLE products (
    product_id SERIAL,
    product_name varchar(10)
);

-- euquivalet with

CREATE SEQUENCE products_id_seq AS integer;

CREATE TABLE products (
    product_id integer NOT NULL DEFAULT nextval('products_id_seq'),
    product_name varchar(10)
);

ALTER SEQUENCE products_id_seq OWNED BY products.product_id;
```



PostgreSQL Data Types

- Character Types
  - Variable-length with limit: **CHARACTER VARYING(n)**, **VARCHAR(n)**
  - Fixed-length, blank padded: **CHARACTER(n)**, **CHAR(n)**
  - Variable unlimited length: **TEXT**
- Binary Data Types: **BYTEA**
- Date/Time Types
  - **TIMESTAMP** – with or without precision and/or time zone
  - **DATE**
  - **TIME** – with or without precision and/or time zone
  - **INTERVAL**
- Boolean Type: **BOOLEAN**
  - Values: **TRUE|FALSE**, **0|1**, **YES|NO**, **ON|OFF**



PostgreSQL Data Types

- Enumerated Types
  - Enumerated (**ENUM**) types are data types that comprise a static, ordered set of values

```
 1 CREATE TYPE happiness AS ENUM ('happy', 'very happy', 'ecstatic');
 2
 3 CREATE TABLE holidays (
 4     num_weeks integer,
 5     happiness happiness
 6 );
 7
 8 INSERT INTO holidays(num_weeks,happiness) VALUES (4, 'happy');
 9 INSERT INTO holidays(num_weeks,happiness) VALUES (6, 'very happy');
10 INSERT INTO holidays(num_weeks,happiness) VALUES (8, 'ecstatic');
11
12 SELECT num_weeks
13 FROM holidays
14 WHERE happiness = (SELECT MIN(happiness) FROM holidays);
15
16 SELECT holidays.num_weeks FROM holidays
17   WHERE holidays.happiness = 'happy';
```



PostgreSQL Data Types

- Geometric Types

Name	Description	Representation
point	Point on a plane	(x,y)
line	Infinite line	{A,B,C}
lseg	Finite line segment	((x1,y1),(x2,y2))
box	Rectangular box	((x1,y1),(x2,y2))
path	Closed path (similar to polygon)	((x1,y1),...)
path	Open path	[(x1,y1),...]
polygon	Polygon (similar to closed path)	((x1,y1),...)
circle	Circle	<(x,y),r> (center point and radius)



PostgreSQL
Geometric Types

```sql
SELECT area(circle '<(0,0), 3>');

SELECT area(box '((2,3),(-2,-3))');

SELECT center(circle '((2,3),8)');

SELECT center(box '((2,3),(-2,-3))');

SELECT diameter(circle '((2,3),8)');

SELECT height(box '((2,3),(-2,-3))');

SELECT isclosed(path '((2,3),(1,1),(2,2))');

SELECT isopen(path '((2,3),(1,1),(2,2))');

SELECT length(lseg '((2,3),(5,5))');

SELECT npoints(path '[(1,1),(2,2),(3,3)]');

SELECT width(box '((2,3),(5,8))');
```



PostgreSQL Data Types

- Network Address Types

Name	Description
cidr	IPv4 and IPv6 networks
inet	IPv4 and IPv6 hosts and networks
macaddr	MAC addresses
macaddr8	MAC addresses (EUI-64 format)

- The essential difference between inet and cidr data types is that
  - **inet** accepts values with nonzero bits to the right of the netmask
  - **cidr** does not accepts values with nonzero bits to the right of the netmask

- For example, 192.168.0.1/24 is valid for **inet** but not for **cidr**.



PostgreSQL Data Types

- Bit String Types
  - Are strings of 1's and 0's
  - Can be used to store or visualize bit masks
  - **bit(n)** - bit string type data must match the length **n** exactly
  - **bit varying(n)** - bit string type data of variable length up to the maximum length **n**

- UUID Type stores Universally Unique Identifiers (UUID)



PostgreSQL
Text Search Data Types

- Text Search Types
  - PostgreSQL provides two data types that are designed to support full text search
  - The **tsvector** type represents a document in a form optimized for text search
  - The **tsquery** type similarly represents a text query



PostgreSQL
XML Types

- XML Type can be used to store XML data
- **xmlcomment(text)** - creates an XML value containing an XML comment with the specified text as content.
- **xmlconcat(xml[, ...])** - concatenates a list of individual XML values to create a single value containing an XML content fragment (NULL values are omitted)
- **xmlelement(name name [, xmlattributes(value [AS attname] [, ...])] [, content, ...])** - creates an XML element with the given name, attributes, and content
- **xmlforest(content [AS name] [, ...])** - produces an XML forest (sequence) of elements using the given names and content.



PostgreSQL
XML Types

- **xmlpi(name target [, content])** - creates an XML processing instruction
- **xmlroot(xml, version text | no value [, standalone yes|no|no value])** - alters the properties of the root node of an XML value
- **xmlagg(xml)** - concatenates the input values to the aggregate function call (the concatenation occurs across rows)
- **xpath(xpath, xml [, nsarray])** - evaluates the XPath expression xpath (a text value) against the XML value xml



PostgreSQL XML Types

- The xmltable function produces a table based on the given XML value, an XPath filter to extract rows, and a set of column definitions

```
xmltable(
        [XMLNAMESPACES(namespace uri AS namespace name
                                        [, ...]), ]
         row_expression PASSING [BY { REF | VALUE }]
                document_expression [BY { REF | VALUE }]
         COLUMNS
                name { type [PATH column_expression]
                      [DEFAULT default_expression] [NOT NULL | NULL]
                      | FOR ORDINALITY }
                [, ...]
)
```



PostgreSQL Data Types

```
SELECT XMLPARSE (DOCUMENT '<?xml version="1.0"?>
    <book>
        <title>Manual</title>
        <chapters>
            <chapter>I PostgreSQL Architecture</chapter>
            <chapter>II PostgreSQL SQL</chapter>
            <chapter>III PostgreSQL PL/pgSQL</chapter>
        </chapters>
    </book>');

SELECT XMLPARSE (CONTENT '<foo>bar</foo><bar>foo</bar>');
```



PostgreSQL Data Types

```sql
CREATE TABLE families(
    family_id SERIAL,
    family_tree XML
);

INSERT INTO families(family_tree) VALUES('<family>
    <mother>Hanna</mother>
    <father>Tom</father>
    <children>
        <child gender="male">John</child>
        <child gender="female">Mary</child>
        <child gender="female">Sue</child>
        <child gender="male">Tim</child>
    </children>
</family>');

SELECT XPATH('//child/text()', family_tree)
FROM FAMILIES;

SELECT XPATH('//@gender', family_tree)
FROM FAMILIES;

SELECT UNNEST(XPATH('//@gender', family_tree))
FROM FAMILIES;

```



PostgreSQL Data Types

```sql
WITH data as (
    SELECT
        UNNEST(XPATH('//child/text()', family_tree)) child_name
    FROM FAMILIES
)
SELECT child_name
FROM data
WHERE child_name::varchar like 'J%';

SELECT
    UNNEST(XPATH('//child[contains(.,"J" )]/text()', family_tree)) child_name
FROM FAMILIES;

SELECT xt.child_name, xt.child_gender
FROM families CROSS JOIN
    xmltable('//child[@gender="male"]'
                passing families.family_tree
                columns
                    id FOR ORDINALITY,
                    child_name   varchar PATH 'text()',
                    child_gender varchar PATH '@gender') as xt;
```



PostgreSQL
JSON Data Type

- JSON Types can be used to store JSON data
  - The **json** data type
    - Stores an exact copy of the input text
    - Processing functions must reparse on each execution
  - The **jsonb** data type
    - Is stored in a decomposed binary format
    - Adds a conversion overhead
      - Makes adding values slower
      - Makes processing faster (since no reparsing is needed)
    - Can be used to test Containment (@>) and Existence (?)
  - Multiple functions for creating and processing JSON data types, e.g.
    - **to_json(anyelement)**
    - **to_jsonb(anyelement)**
    - **jsonb_path_query(target jsonb, path jsonpath [, vars jsonb [, silent bool]])**



PostgreSQL
JSON Data Type

Operator	Right Operand Type	Return type	Description
->	int	json or jsonb	Get JSON array element (indexed from zero, negative integers count from the end)
->	text	json or jsonb	Get JSON object field by key
->>	int	text	Get JSON array element as text
->>	text	text	Get JSON object field as text
#>	text[]	json or jsonb	Get JSON object at the specified path
#>>	text[]	text	Get JSON object at the specified path as text



PostgreSQL
JSON Data Type

```
 1 SELECT '[{"a":"foo"},{"b":"bar"},{"c":"baz"}]'::json->2;
 2
 3 SELECT '{"a": {"b":"foo"}}'::json->'a';
 4
 5 SELECT '[1,2,3]'::json->>2;
 6
 7 SELECT '{"a":1,"b":2}'::json->>'b';
 8
 9 SELECT '{"a": {"b":{"c": "foo"}}}'::json#>'{a,b}';
10
11 SELECT '{"a":[1,2,3],"b":[4,5,6]}'::json#>>'{a,2}';
```



PostgreSQL
JSON Data Type

Operator	Right Operand Type	Description
@>	jsonb	Does the left JSON value contain the right JSON path/value entries at the top level?
<@	jsonb	Are the left JSON path/value entries contained at the top level within the right JSON value?
?	text	Does the string exist as a top-level key within the JSON value?
?|	text[]	Do any of these array strings exist as top-level keys?
?&	text[]	Do all these array strings exist as top-level keys?
||	jsonb	Concatenate two jsonb values into a new jsonb value
-	text	Delete key/value pair or string element from left operand. Key/value pairs are matched based on their key value.
-	text[]	Delete multiple key/value pairs or string elements from left operand. Key/value pairs are matched based on their key value.
-	integer	Delete the array element with specified index (Negative integers count from the end). Throws an error if top level container is not an array.
#-	text[]	Delete the field or element with specified path (for JSON arrays, negative integers count from the end)
@?	jsonpath	Does JSON path return any item for the specified JSON value?
@@	jsonpath	Returns the result of JSON path predicate check for the specified JSON value. Only the first item of the result is taken into account. If the result is not Boolean, then null is returned.



PostgreSQL
JSON Data Type

```sql
SELECT '{"a":1, "b":2}'::jsonb @> '{"b":2}'::jsonb;

SELECT '{"b":2}'::jsonb <@ '{"a":1, "b":2}'::jsonb;

SELECT '{"a":1, "b":2}'::jsonb ? 'b';

SELECT '{"a":1, "b":2, "c":3}'::jsonb ?| array['b', 'c'];

SELECT '["a", "b"]'::jsonb ?& array['a', 'b'];

SELECT '["a", "b"]'::jsonb || '["c", "d"]'::jsonb;

SELECT '{"a": "b"}'::jsonb - 'a';

SELECT '{"a": "b", "c": "d"}'::jsonb - '{a,c}'::text[];

SELECT '["a", "b"]'::jsonb - 1;

SELECT '["a", {"b":1}]'::jsonb #- '{1,b}';

SELECT '{"a":[1,2,3,4,5]}'::jsonb @? '$.a[*] ? (@ > 2)';

SELECT '{"a":[1,2,3,4,5]}'::jsonb @@ '$.a[*] > 2';
```



PostgreSQL
JSON Data Type

```sql
-- Simple scalar/primitive values contain only the identical value:
SELECT '"foo"'::jsonb @> '"foo"'::jsonb;

-- The array on the right side is contained within the one on the left:
SELECT '[1, 2, 3]'::jsonb @> '[1, 3]'::jsonb;

-- Order of array elements is not significant, so this is also true:
SELECT '[1, 2, 3]'::jsonb @> '[3, 1]'::jsonb;

-- Duplicate array elements don't matter either:
SELECT '[1, 2, 3]'::jsonb @> '[1, 2, 2]'::jsonb;

-- The object with a single pair on the right side is contained
-- within the object on the left side:
SELECT '{"product": "PostgreSQL", "version": 9.4, "jsonb": true}'::jsonb @> '{"version":
9.4}'::jsonb;

-- The array on the right side is not considered contained within the
-- array on the left, even though a similar array is nested within it:
SELECT '[1, 2, [1, 3]]'::jsonb @> '[1, 3]'::jsonb;  -- yields false

-- But with a layer of nesting, it is contained:
SELECT '[1, 2, [1, 3]]'::jsonb @> '[[1, 3]]'::jsonb;

-- Similarly, containment is not reported here:
SELECT '{"foo": {"bar": "baz"}}'::jsonb @> '{"bar": "baz"}'::jsonb;  -- yields false

-- A top-level key and an empty object is contained:
SELECT '{"foo": {"bar": "baz"}}'::jsonb @> '{"foo": {}}'::jsonb;

-- String exists as array element:
SELECT '["foo", "bar", "baz"]'::jsonb ? 'bar';

-- String exists as object key:
SELECT '{"foo": "bar"}'::jsonb ? 'foo';

-- Object values are not considered:
SELECT '{"foo": "bar"}'::jsonb ? 'bar';  -- yields false

-- As with containment, existence must match at the top level:
SELECT '{"foo": {"bar": "baz"}}'::jsonb ? 'bar'; -- yields false

-- A string is considered to exist if it matches a primitive JSON string:
SELECT '"foo"'::jsonb ? 'foo';
```



PostgreSQL
JSON Data Type

```
CREATE TABLE characters (data jsonb);

INSERT INTO characters VALUES ('
{ "name" : "Yksdargortso",
  "id" : 1,
  "sex" : "male",
  "hp" : 300,
  "level" : 10,
  "class" : "warrior",
  "equipment" :
   {
     "rings" : [
       { "name" : "ring of despair",
         "weight" : 0.1
       },
       {"name" : "ring of strength",
        "weight" : 2.4
       }
     ],
     "arm_right" : "Sword of flame",
     "arm_left" : "Shield of faith"
   }
}');
```



PostgreSQL
JSON Data Type

```sql
SELECT jsonb_path_query(data, '$.name')
FROM characters;

SELECT jsonb_path_query(data, '$.equipment.rings[0].*') AS data
FROM characters;

SELECT jsonb_path_query(data, '$.equipment.rings[*].weight.floor()') AS weight
FROM characters;

SELECT jsonb_path_query(data, '$.equipment.rings[*] ? (@.weight > 1)')->'name' AS name
FROM characters;
```



PostgreSQL
Array Data Type

- PostgreSQL allows columns of a table to be defined as variable-length multidimensional arrays

- Arrays of any built-in or user-defined base type, enum type, composite type, range type, or domain can be created



PostgreSQL
Array Data Type

```
 1 CREATE TABLE sal_emp (
 2     name            text,
 3     pay_by_quarter  integer[],
 4     schedule        text[][]
 5 );
 6
 7 INSERT INTO sal_emp
 8 VALUES ('Bill',
 9     '{12000, 10000, 11000, 13000}',
10     '{{"meeting", "lunch"}, {"training", "presentation"}}');
11
12 INSERT INTO sal_emp
13     VALUES ('Carol',
14     ARRAY[20000, 25000, 25000, 25000],
15     ARRAY[['breakfast', 'consulting'], ['meeting', 'lunch']]);
16
17
18 SELECT * FROM sal_emp;
```



PostgreSQL
Array Data Type

```sql
SELECT name FROM sal_emp WHERE pay_by_quarter[1] <> pay_by_quarter[2];

SELECT pay_by_quarter[3] FROM sal_emp;

SELECT schedule[1:2][1:1] FROM sal_emp WHERE name = 'Bill';

UPDATE sal_emp SET pay_by_quarter = '{25000,25000,27000,27000}'
    WHERE name = 'Carol';

UPDATE sal_emp SET pay_by_quarter = ARRAY[25000,25000,27000,27000]
    WHERE name = 'Carol';

UPDATE sal_emp SET pay_by_quarter[4] = 15000
    WHERE name = 'Bill';

UPDATE sal_emp SET pay_by_quarter[1:2] = '{27000,27000}'
    WHERE name = 'Carol';
```



PostgreSQL
Composite Data Type

- A *composite type* represents the structure of a row or record
- It is essentially just a list of field names and their data types
- PostgreSQL allows composite types to be used in many of the same ways that simple types can be used



PostgreSQL
Composite Data Type

```
 1 CREATE TYPE inventory_item AS (
 2     name            text,
 3     supplier_id     integer,
 4     price           numeric
 5 );
 6
 7 CREATE TABLE items (
 8     item_id   SERIAL,
 9     item      inventory_item,
10     count     integer
11 );
12
13 INSERT INTO items(item, count)
14     VALUES (ROW('fuzzy dice', 42, 1.99), 1000);
15 INSERT INTO items(item, count)
16     VALUES (ROW('extra fuzzy dice', 42, 10.99), 999);
17 INSERT INTO items(item, count)
18     VALUES (ROW(NULL, 42, 5.99), 500);
19 INSERT INTO items(count) VALUES (500);
```



PostgreSQL
Composite Data Type

```sql
SELECT (item).name FROM items
    WHERE (item).price > 9.99;

UPDATE items
    SET item = ROW('not that fuzzy dice', 42, 2.99)
    WHERE item_id = 4;

UPDATE items
    SET item.name = 'simple dice',
        item.price = (item).price + 5
    WHERE item_id = 3;
```